ДЕПАРТАМЕНТ ОБРАЗОВАНИЯ И НАУКИ ГОРОДА МОСКВЫ

ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ПРОФЕССИОНАЛЬНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
«Колледж автоматизации и информационных технологий № 20»

КУРСОВОЙ ПРОЕКТ

Тема Проектирование и разработка приложения для ведения управления заявками в игровом клубе
(наименование темы)

Код и наименование специальности 09.02.07 Информационные системы и программирование
Группа ИСП222

Студент _____/ Егоров Кирилл Денисович
(подпись) (фио)

Руководитель _____/Лисавина Алёна Вадимовна
(подпись) (фио)

Москва, 2026

ДЕПАРТАМЕНТ ОБРАЗОВАНИЯ И НАУКИ ГОРОДА МОСКВЫ

ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ПРОФЕССИОНАЛЬНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
«Колледж автоматизации и информационных технологий № 20»

РАССМОТРЕНО:
на заседании ППО
«Информационные технологии»
от «09» февраля 2026 г.
Протокол № 4
Председатель ППО
_____/Кузьмина О.Б.
(подпись) (Ф.И.О.)

Руководитель по курсовому проекту
_____/ Лисавина А.В.
(подпись) (Ф.И.О.)

УТВЕРЖДАЮ:
Заместитель директора по качеству образования
_____/ Глазков П.А.
(подпись) (Ф.И.О.)
«___»_____ 2026 г.

ЗАДАНИЕ
на курсовой проект

Студенту(ке) Егорову Кириллу Денисовичу
(Ф.И.О. полностью)

Тема Проектирование и разработка приложения для ведения управления заявками в игровом клубе
(наименование темы)

Содержание курсового проекта	**Сроки исполнения задания**	**Примечание**
ВВЕДЕНИЕ (~ 5%)	11.03.2026- 16.03.2026	
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ВЫБОР ТЕХНОЛОГИЧЕСКОГО СТЕКА (~ 25-35 %) 1.1 Анализ предметной области 1.1.1 Описание бизнес-процессов 1.1.2 Выявление проблем и обоснование необходимости автоматизации 1.1.3 Анализ существующих решений	17.03.2026- 23.03.2026	
1.2 Формулировка требований к системе 1.2.1 Функциональные требования 1.2.2 Нефункциональные требования	24.03.2026- 30.03.2026	
1.3 Выбор и обоснование стека технологий	31.03.2026	
2 ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА СИСТЕМЫ (~ 40-50 %) 2.1 Проектирование 2.1.1 Проектирование базы данных 2.1.2 Диаграмма вариантов использования	01.04.2026 08.04.2026	
2.2 Разработка 2.2.1 Реализация модели базы данных 2.2.2 Разработка приложения 2.2.2.1 Реализация клиентской части 2.2.2.2 Разработка серверной части	09.04.2026 20.04.2026	

2.3 Организация хранения и доступа к файлам посредством репозитория	21.04.2026	
2.4 Тестирование системы	22.04.2026	
ЗАКЛЮЧЕНИЕ (~ 8-10 %)	23.04.2026	
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	24.04.2026	

Дата выдачи задания «09» февраля 2026 г.

Задание получил _____ «09» февраля 2026 г.

(подпись студента)

СОДЕРЖАНИЕ

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ 6

ВВЕДЕНИЕ..... 7

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ВЫБОР ОПТИМАЛЬНОГО ТЕХНОЛОГИЧЕСКОГО СТЕКА..... 9

1.1 Анализ предметной области..... 9

1.1.1 Описание бизнес-процессов 9

1.1.2 Выявление проблем и обоснование необходимости автоматизации 10

1.1.3 Анализ существующих решений..... 11

1.2 Формулировка требований к системе 12

1.2.1 Функциональные требования 13

1.2.2 Нефункциональные требования 13

1.3 Выбор и обоснование стека технологий..... 14

2 ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА СИСТЕМЫ..... 17

2.1 Проектирование..... 17

2.1.1 Проектирование базы данных..... 17

2.1.2 Диаграмма вариантов использования 18

2.2 Разработка 20

2.2.1 Реализация модели базы данных 20

2.2.2 Разработка приложения 21

2.2.2.1 Реализация клиентской части 21

2.2.2.2 Разработка серверной части..... 25

2.3 Организация хранения и доступа к файлам посредством репозитория 28

2.4 Тестирование системы..... 29

ЗАКЛЮЧЕНИЕ 33

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ..... 35

ПРИЛОЖЕНИЯ 38

ПРИЛОЖЕНИЕ А. Программный код страницы «Авторизация» 38

ПРИЛОЖЕНИЕ Б. Программный код формы «Регистрация» 40

ПРИЛОЖЕНИЕ В. Программный код формы «Личный кабинет клиента» ... 41

ПРИЛОЖЕНИЕ Г. Программный код формы «Личный кабинет менеджера»43
ПРИЛОЖЕНИЕ Д. SQL-скрипт создания базы данных и заполнения тестовых данных 46

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ

ER – Entity-Relationship (диаграмма сущность-связь).

CRM – Customer Relationship Management, система управления взаимоотношениями с клиентами.

SQL – Structured Query Language, структурированный язык запросов.

LINQ – Language Integrated Query, встроенный язык запросов в C#.

WPF – Windows Presentation Foundation, платформа разработки интерфейсов.

GIT – распределенная система контроля версий.

MySQL – My Structured Query Language, реляционная система управления базами данных (СУБД) с открытым исходным кодом, основанная на языке SQL.

Windows Forms – это платформа пользовательского интерфейса для создания классических приложений Windows.

ВВЕДЕНИЕ

Игровые клубы занимают устойчивую нишу в сфере компьютерного досуга: они предоставляют доступ к профессиональному оборудованию и высокоскоростному интернету для тех, кто не имеет возможности или желания организовать игровое место самостоятельно. Вместе с постоянным потоком клиентов появляется необходимость вести учет их обращений – бронирований, жалоб, запросов на продление времени [1, с. 22–25]. Отсутствие удобного инструмента для этого приводит к потере данных, путанице и снижению качества обслуживания.

Специализированные CRM-системы[1] позволяют централизовать учет заявок: все обращения фиксируются в единой базе, история сохраняется, статус каждой заявки прозрачен. Тем не менее большинство готовых решений ориентированы на средний и крупный бизнес, содержат избыточный функционал и требуют значительных затрат на внедрение и обучение персонала. Для небольшого игрового клуба такой подход экономически нецелесообразен.

Данный проект направлен на разработку собственного приложения для управления заявками, адаптированного под условия небольшого клуба. Система предусматривает два типа пользователей: клиент, который самостоятельно создает заявки и отслеживает их статус, и менеджер, который просматривает все обращения, изменяет их статус, редактирует и при необходимости удаляет записи.

Объект исследования – деятельность игрового клуба, связанная с приемом и обработкой обращений клиентов. Предмет исследования – методы автоматизации работы с заявками: их создание, хранение в реляционной базе данных и управление состоянием посредством программного интерфейса.

---

[1] CRM-система – это функциональный инструмент, который помогает выстраивать и поддерживать отношения с клиентами. [Электронный ресурс] – URL: https://crm.ru/blog/crm-chto-eto-takoe/?utm_referrer=https%3A%2F%2Fwww.google.com%2F#anchor-2 Режим доступа: свободный (дата обращения: 12.03.2026).

Цель проекта – разработать приложение для автоматизации учета заявок в игровом клубе, обеспечивающее удобную работу для сотрудников и клиентов заведения.

Для достижения поставленной цели необходимо решить следующие задачи:

1) Проанализировать бизнес-процессы игрового клуба и выявить проблемы существующего подхода к учету заявок;

2) Изучить аналогичные программные решения и определить их недостатки применительно к условиям небольшого клуба;

3) Сформулировать функциональные и нефункциональные требования к разрабатываемой системе;

4) Спроектировать структуру базы данных и диаграмму вариантов использования;

5) Реализовать приложение;

6) Провести тестирование системы и проверить корректность выполнения всех функций.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ВЫБОР ОПТИМАЛЬНОГО ТЕХНОЛОГИЧЕСКОГО СТЕКА

1.1 Анализ предметной области

1.1.1 Описание бизнес-процессов

В игровом клубе взаимодействуют два типа пользователей: клиент и менеджер. Клиент пользуется услугами клуба и при необходимости оставляет обращение: забронировать рабочее место, сообщить о неполадке с оборудованием или продлить игровое время. Менеджер принимает поступившие заявки и обрабатывает их в порядке очередности.

При отсутствии специализированного программного обеспечения рабочий процесс складывается следующим образом: клиент обращается устно или передает просьбу в письменном виде, менеджер фиксирует ее в блокноте или таблице Excel. При смене персонала восстановить полную картину обращений затруднительно: часть данных остается необработанной, часть теряется. Это приводит к задержкам в обслуживании и снижает удовлетворенность клиентов.

В разрабатываемой системе процесс организован иначе. Клиент авторизуется в приложении под своей учетной записью и самостоятельно создает заявку, указывая название и описание. Статус «Новая» присваивается автоматически, дата создания фиксируется системой. Менеджер видит все поступившие заявки, может отфильтровать их по статусу или найти обращения конкретного клиента. По мере обработки статус меняется: «В работе» или «Завершена».

При необходимости менеджер добавляет заявку вручную – например, если клиент обратился устно. Также предусмотрена возможность редактирования и удаления записей.

Вся информация хранится в реляционной базе данных: оба участника работают с актуальными данными, исключая необходимость устного согласования или поиска записей в бумажных журналах [3, с. 36–40].

Описанная организация работы служит основой для выявления проблем существующего подхода и формулирования требований к автоматизированной системе.

1.1.2 Выявление проблем и обоснование необходимости автоматизации

Игровые клубы, в которых заявки до сих пор фиксируются вручную, на практике сталкиваются с рядом классических затруднений. Анализ текущих процессов показал, что подобные проблемы носят системный характер и напрямую воздействуют на качество обслуживания клиентов [2, с. 95–100].

В первую очередь, стоит отметить риск потери данных. Записи, сделанные в блокнотах или отдельных файлах, не защищены от случайного удаления или потери данных при передаче смены следующему сотруднику. Не менее важной проблемой являются ошибки при вводе: опечатки в именах, неточные даты или неверно указанные номера игровых мест клиентов. Все это приводит к путанице и требует дополнительного времени на исправление.

Также отсутствует наглядное представление о текущем состоянии заявок. Без единой системы сложно определить, какие обращения уже обработаны, а какие находятся в работе. При увеличении количества клиентов происходят задержки и снижение эффективности работы персонала. Большой список записей затрудняет поиск необходимых данных среди рукописных заметок или объемных таблиц.

Отдельно стоит выделить отсутствие разграничения прав доступа. С ручным учетом любой сотрудник может внести изменения или удалить информацию. Помимо этого, клиент оказывается в ситуации, когда он зависим от менеджера: для уточнения статуса заявки ему необходимо обращаться напрямую, что создает лишнюю нагрузку на персонал.

Внедрение специализированного программного обеспечения позволяет многократно упростить и упорядочить работу с заявками. Хранение данных в реляционной базе обеспечивает их сохранность и целостность. Часть информации фиксируется автоматически, такая как дата создания или текущий

статус. Сотрудники получают доступ к актуальному списку заявок с возможностью поиска и фильтрации. Клиент может самостоятельно отслеживать статус своего обращения, не обращаясь к менеджеру.

Таким образом, разработка собственного приложения является обоснованным решением с точки зрения повышения эффективности информационной системы [4, с. 94–101]

1.1.3 Анализ существующих решений

До начала разработки был проведен анализ существующих программных продуктов, частично подходящих для управления заявками в небольшом заведении.

Bitrix24 – многофункциональная CRM-платформа с модулем задач и заявок. Бесплатная версия существует, однако ее возможности ограничены: для полноценной работы с клиентами необходим платный тариф, стоимость которого для малого бизнеса ощутима. Интерфейс платформы содержит большое количество функций, освоение которых без предварительного обучения затруднительно.

1С: Предприятие – отечественная платформа, зарекомендовавшая себя в области бухгалтерского учета и документооборота. Для управления заявками она избыточна: адаптация под конкретную задачу требует привлечения квалифицированного 1С-программиста, а стоимость лицензий и обучения персонала существенно увеличивает итоговые затраты.

Сравнение характеристик рассмотренных систем приведено в таблице 1.

Таблица 1 – Сравнение существующих решений

Характеристика	Bitrix24	1С: Предприятие
Стоимость	Бесплатная версия с ограничениями; платные тарифы – средние и выше	Платная лицензия; высокая стоимость внедрения
Сложность освоения	Средняя; разветвленный интерфейс требует времени на изучение	Высокая; персонал нуждается в специальном обучении
Настройка под задачи	Гибкая, но требует специалиста	Только через программиста 1С
Работа с заявками	Реализована, но перегружена сопутствующим функционалом	Нет готового модуля; требуется доработка
Пригодность для малого клуба	Частичная	Не подходит

Интерфейсы рассмотренных платформ представлены на рисунках 1 и 2.

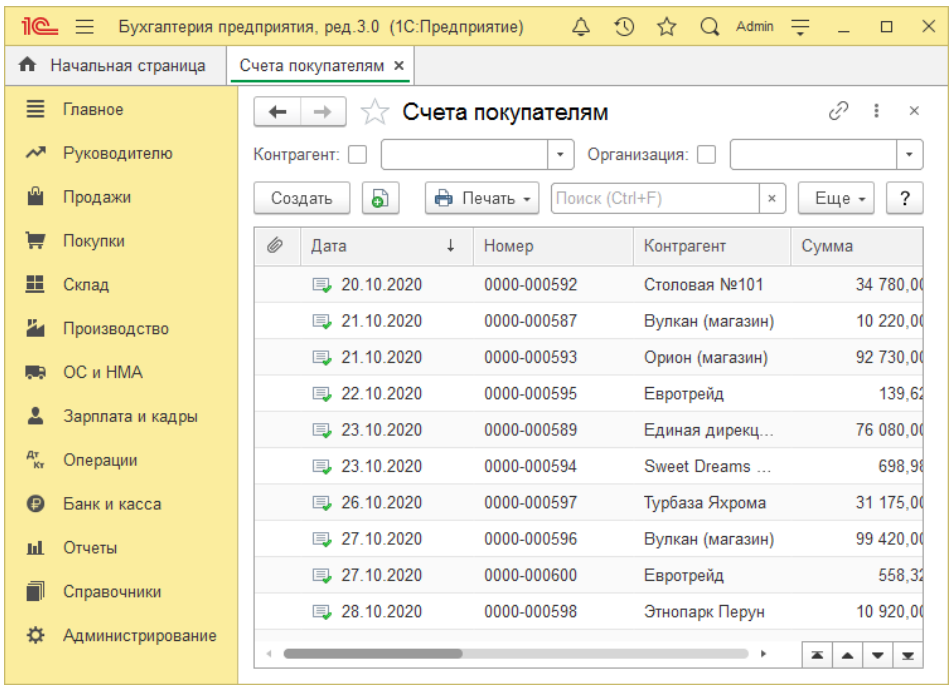

Рисунок 1 – Интерфейс платформы 1С: Предприятие

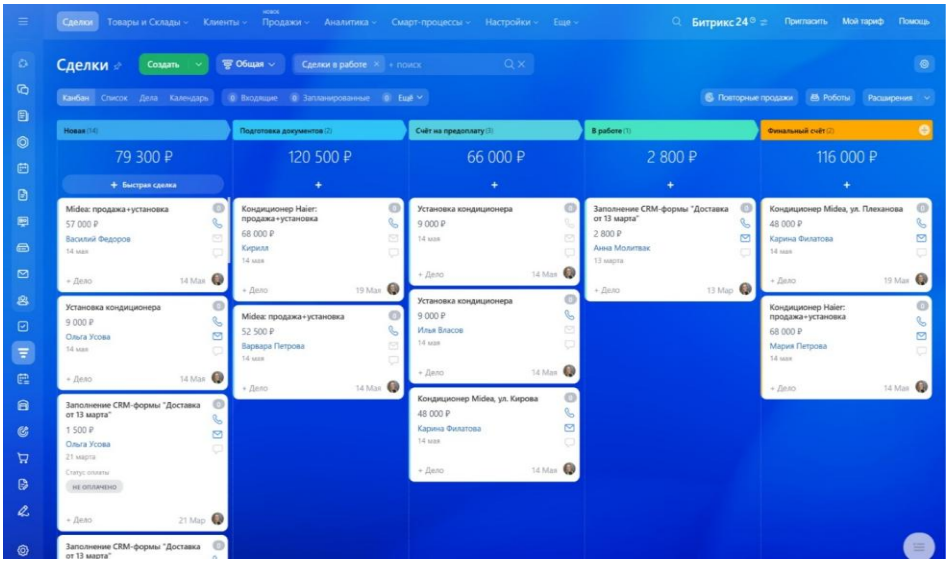

Рисунок 2 – Интерфейс приложения Bitrix24

Проведенный анализ показывает, что ни одно из рассмотренных решений не является оптимальным для небольшого игрового клуба. Именно поэтому было принято решение разработать собственное приложение - с минимально необходимым функционалом, без избыточных возможностей и затрат на внедрение.

1.2 Формулировка требований к системе

На основе анализа предметной области и выявленных проблем были сформулированы требования к разрабатываемой системе. Требования разделены на две категории: функциональные, определяющие перечень действий,

которые система должна выполнять, и нефункциональные, задающие требования к качеству ее работы.

1.2.1 Функциональные требования

Регистрация и авторизация. Пользователь создает учетную запись, указывая логин, пароль и роль. При входе система проверяет корректность введенных данных и соответствие выбранной роли – в случае несоответствия доступ не предоставляется.

Функции клиента. Клиент работает исключительно со своими заявками. Он может создать новую заявку, указав название и описание, просмотреть список своих обращений, а также использовать поиск по названию и фильтрацию по статусу.

Функции менеджера. Менеджер имеет доступ к заявкам всех клиентов. Он может добавить заявку от имени выбранного клиента, изменить статус существующей заявки (Новая, В работе, Завершена), отредактировать ее содержимое или удалить запись. Предусмотрена фильтрация по клиенту и по статусу.

Хранение данных. Все записи хранятся в базе данных MySQL[2]. Данные сохраняются между сессиями и не зависят от момента завершения работы с приложением.

1.2.2 Нефункциональные требования

Система должна работать быстро – поиск, загрузка и фильтрация не тормозят даже при типичном объеме данных небольшого клуба. База данных хранит все без потерь – никакие программные сбои не должны приводить к порче или исчезновению информации.

Каждый пользователь видит только свои данные – клиент не может залезть в чужие записи или что-то там поменять, а менеджер при этом имеет расширенный доступ в рамках своей роли. Интерфейс устроен так, что

---

[2] MySQL (My Structured Query Language) – реляционная система управления базами данных (СУБД) с открытым исходным кодом, основанная на языке SQL.[Электронный ресурс] – URL: https://selectel.ru/blog/mysql / Режим доступа: свободный (дата обращения: 09.04.2026).

разобраться можно сразу, без всяких инструкций и обучения – основные функции понятны с первого взгляда.

Код написан так, чтобы его можно было спокойно дорабатывать – добавить новую функцию или починить баг, не трогая все остальное.

1.3 Выбор и обоснование стека технологий

Выбор технологий определялся требованиями к приложению: простота настольного клиента для Windows, работа с реляционной базой данных и возможность реализации всего функционала в разумные сроки.

Язык программирования – C#. Рассматривались также Java и Python. Java обеспечивает высокую производительность, однако для реализации настольного приложения Windows требует значительно большего объема кода. Python удобен для прототипирования, но при интенсивной работе с базой данных его производительность ниже. C# хорошо подходит для разработки настольных приложений Windows, имеет современный синтаксис, поддерживает Language Integrated Query (LINQ[3]) для удобной работы с данными и нативно интегрирован с Windows Forms[4]. Сравнение языков программирования приведено в таблице 2.

Таблица 2 – Сравнение языков программирования

Параметр	C#	Java	Python
Простота освоения	Высокая	Средняя	Высокая
Производительность	Высокая	Высокая	Средняя
Объем кода	Умеренный	Большой	Небольшой
Поддержка Windows Forms	Да	Нет	Ограниченно

Среда разработки – Microsoft Visual Studio. Предоставляет удобный редактор кода, встроенный отладчик и визуальный конструктор форм, что существенно ускоряет разработку интерфейсов Windows Forms. Альтернативные

[3] LINQ (Language Integrated Query) – это набор методов для работы с коллекциями. [Электронный ресурс] – URL: https://habr.com/ru/articles/836752/ Режим доступа: свободный (дата обращения: 31.03.2026).

[4] Windows Forms (WinForms) – это платформа пользовательского интерфейса для создания классических приложений Windows. [Электронный ресурс] – URL: https://learn.microsoft.com/ru-ru/dotnet/desktop/winforms/overview/ Режим доступа: свободный (дата обращения: 31.03.2026).

среды рассматривались, однако для данного стека Visual Studio остается наиболее логичным выбором.

Технология пользовательского интерфейса – Windows Forms. WPF[5] обеспечивает более гибкий визуальный стиль, однако требует знания языка разметки XAML и значительных временных затрат на освоение. ASP.NET предназначен для веб-приложений, что не соответствует задаче. Windows Forms позволяет оперативно создавать необходимые элементы управления в визуальном редакторе и сразу переходить к написанию логики. Сравнение технологий пользовательского интерфейса приведено в таблице 3.

Таблица 3 – Сравнение технологий пользовательского интерфейса

Параметр	Windows Forms	WPF	ASP.NET
Простота освоения	Высокая	Низкая	Средняя
Скорость разработки	Высокая	Средняя	Средняя
Тип приложения	Настольное (Windows)	Настольное (Windows)	Веб-приложение
Визуальный редактор	Да	Да	Нет

MySQL система бесплатна, хорошо интегрируется с C# посредством библиотеки MySql.Data, а MySQL Workbench предоставляет удобный инструмент для управления базой данных и написания SQL-запросов[6]. PostgreSQL предлагает расширенные возможности, однако для данного проекта они избыточны. Oracle является платной системой с высоким порогом входа. Сравнение СУБД приведено в таблице 4.

Таблица 4 – Сравнение СУБД

Параметр	MySQL	PostgreSQL	Oracle
Стоимость	Бесплатная	Бесплатная	Платная
Производительность	Высокая	Высокая	Очень высокая
Сложность освоения	Низкая	Средняя	Высокая
Интеграция с C#	Хорошая	Хорошая	Требует настройки

[5] WPF (Windows Presentation Foundation) — это библиотека для создания пользовательских интерфейсов для интеллектуальных клиентских приложений. [Электронный ресурс] – URL: https://professorweb.ru/my/WPF/base_WPF/level1/base_WPF_index.php / Режим доступа: свободный (дата обращения: 31.03.2026).

[6] SQL-запросы — Structured Query Language предназначен для управления базами данных. Чтобы обрабатывать данные из базы, нужно использовать SQL-запросы. . [Электронный ресурс] – URL: https://practicum.yandex.ru/blog/sql-zaprosy-dlya-upravleniya-bazami-dannyh / / Режим доступа: свободный (дата обращения: 09.04.2026).

Для построения диаграмм применялся Microsoft Visio – инструмент с широким набором готовых элементов для создания ER-диаграмм и схем вариантов использования.

Выбранный стек – C#, Windows Forms, MySQL – полностью покрывает требования проекта, не создает избыточной сложности и позволяет сосредоточиться на реализации нужного функционала.

2 ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА СИСТЕМЫ

2.1 Проектирование

Проектирование предшествует разработке и позволяет заранее определить структуру данных, логику их взаимосвязей и набор действий, доступных каждому типу пользователей. Качественно проработанный этап проектирования сокращает количество переделок при написании кода [6, с. 179–190].

На данном этапе были разработаны два документа: ER-диаграмма[7] базы данных и диаграмма вариантов использования. Оба позволили выявить логические несоответствия до начала реализации.

Уже на стадии проектирования стало очевидным, что клиент и менеджер должны работать в независимых интерфейсах с разграниченным набором действий. Это требование отразилось как на структуре базы данных, так и на организации форм приложения.

2.1.1 Проектирование базы данных

Первым шагом проектирования стало определение ключевых объектов системы: пользователи и их заявки. Вокруг этих двух сущностей строится вся логика приложения.

ER-диаграмма, отражающая структуру базы данных, представлена на рисунке 3.

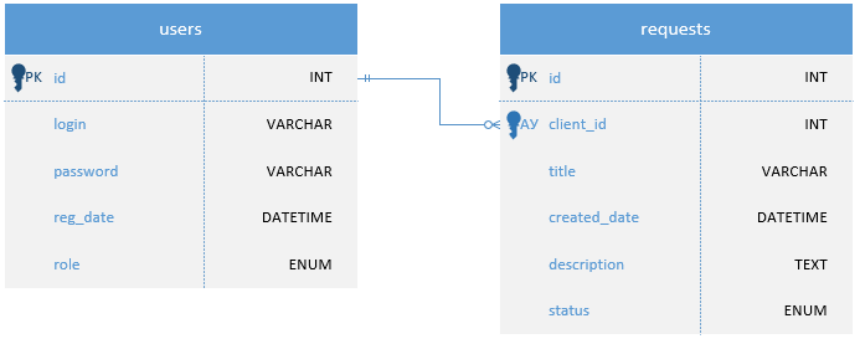

Рисунок 3 – ER-диаграмма базы данных

[7] ER-диаграмма (Entity-Relationship, или диаграмма «сущность-связь») — это схема, которая показывает, какие данные есть в системе и как они связаны друг с другом. [Электронный ресурс] – URL: https://kurshub.ru/journal/blog/chto-takoe-er-diagramma/ Режим доступа: свободный (дата обращения: 01.04.2026).

Таблица users хранит сведения об учетных записях: уникальный идентификатор, login (с ограничением уникальности), password, role и дату регистрации. Поле role объявлено как ENUM с двумя допустимыми значениями – manager и client. От значения этого поля зависит, к какому интерфейсу получит доступ пользователь после авторизации.

Таблица requests хранит заявки: идентификатор, внешний ключ на пользователя, title, description, status и дату создания (created_date). Статус также является ENUM-полем с тремя значениями: New, In_progress, Completed. В приложении статусы отображаются по-русски. При добавлении новой записи статус устанавливается автоматически как New (отображается как «Новая»).

Связь между таблицами реализована по принципу «Один ко многим»: один пользователь может иметь несколько заявок, каждая заявка относится к единственному пользователю. Использование внешнего ключа с условием ON DELETE CASCADE гарантирует, что при удалении пользователя связанные с ним заявки будут удалены автоматически, сохраняя целостность данных.

Структура базы данных спроектирована с учетом принципов нормализации: данные о пользователях и заявках не дублируются, каждая сущность хранится в отдельной таблице. Это упрощает сопровождение системы и исключает избыточность.

2.1.2 Диаграмма вариантов использования

Диаграмма вариантов использования (Use Case) описывает систему со стороны пользователя – показывает, какие действия доступны каждому из акторов, не затрагивая внутреннюю реализацию [7, с. 117–123]. Диаграмма представлена на рисунке 4.

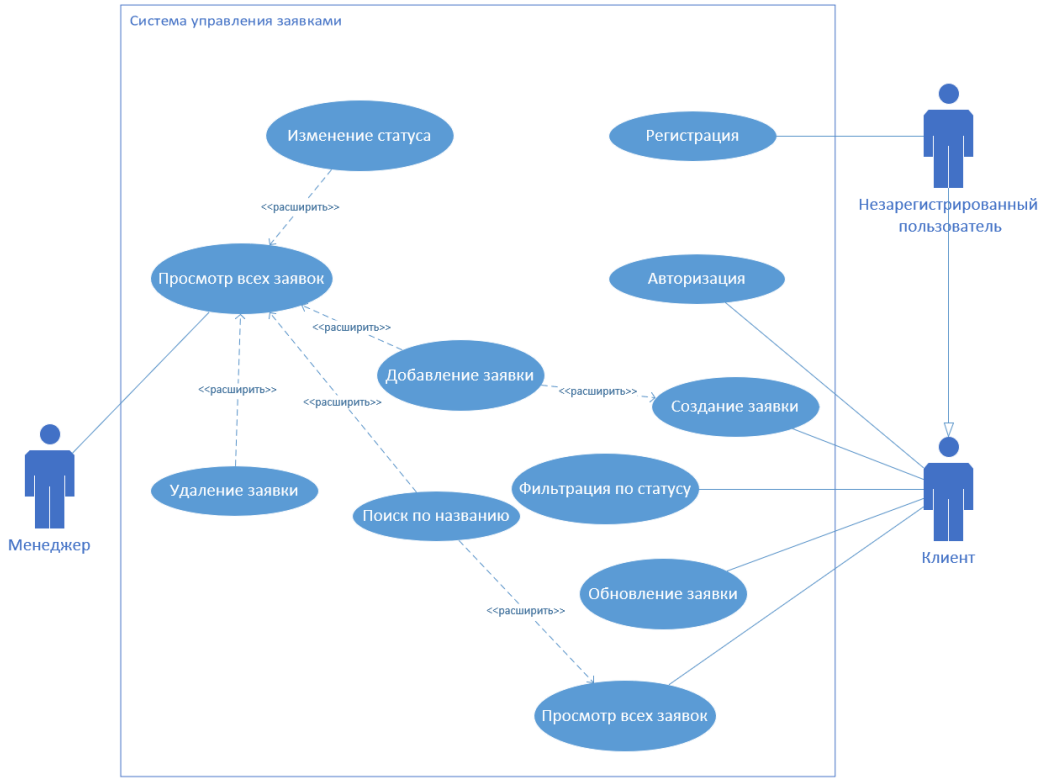

Рисунок 4 – Диаграмма вариантов использования

В системе определены два актора – Клиент и Менеджер. Оба проходят процедуру регистрации и авторизации. Дальнейший набор доступных функций различается.

Клиент может: создать новую заявку, просмотреть список своих обращений, использовать поиск по названию, применить фильтр по статусу. Данные других пользователей клиенту недоступны.

Менеджер может: просматривать заявки всех клиентов, добавлять заявку от имени выбранного клиента, изменять статус существующей заявки, редактировать ее содержимое, удалять запись. Дополнительно доступна фильтрация по клиенту и статусу, а также поиск по названию.

Такое разграничение исключает возможность несанкционированного доступа к чужим данным и четко определяет рабочую область каждого пользователя.

По результатам проектирования были получены документы, достаточные для перехода к реализации: структура базы данных и перечень функций системы.

2.2 Разработка

Реализация велась последовательно: сначала была создана база данных, затем разработаны формы приложения, после чего настроено взаимодействие между интерфейсом и данными. Такой порядок позволил проверить корректность логики на каждом этапе, не откладывая тестирование на финал.

2.2.1 Реализация модели базы данных

База данных создавалась в среде MySQL Workbench с использованием локального подключения. Проект начинался с формирования базы с кодировкой utf8mb4, обеспечивающей корректное хранение кириллических символов. Затем создавались таблицы, после чего в них добавлялись тестовые данные для проверки работы системы. Ниже приведен полный SQL-скрипт, представленный на рисунке 5. Скрипт также приведен в Приложении Д.

```
1  DROP DATABASE IF EXISTS testdb;
2  CREATE DATABASE testdb CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
3  USE testdb;
4
5  CREATE TABLE users (
6      id INT AUTO_INCREMENT PRIMARY KEY,
7      login VARCHAR(50) NOT NULL UNIQUE,
8      password VARCHAR(100) NOT NULL,
9      role ENUM('manager', 'client') NOT NULL DEFAULT 'client',
10     reg_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP
11 );
12
13 CREATE TABLE requests (
14     id INT AUTO_INCREMENT PRIMARY KEY,
15     client_id INT NOT NULL,
16     title VARCHAR(255) NOT NULL,
17     description TEXT,
18     status ENUM('New', 'In_progress', 'Completed') DEFAULT 'New',
19     created_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
20     FOREIGN KEY (client_id) REFERENCES users(id) ON DELETE CASCADE
21 );
22
23 INSERT INTO users (login, password, role) VALUES
24 ('manager', '1234', 'manager'),
25 ('user1', '1234', 'client'),
26 ('user2', '1234', 'client');
27
28 INSERT INTO requests (client_id, title, description, status) VALUES
29 (2, 'Бронирование ПК', 'Хочу забронировать компьютер на 3 часа', 'New'),
30 (2, 'Мышка не работает', 'Мышка на месте 5 не работает', 'In_progress'),
31 (3, 'Продление времени', 'Продлить игровое время на 1 час', 'Completed');
```

Рисунок 5 – Полный SQL скрипт

Снимок экрана результата выполнения скрипта представлен на рисунке 6.

Output

Action Output

#	Time	Action
4	16:38:57	CREATE DATABASE testdb CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci
5	16:38:57	USE testdb
6	16:38:57	CREATE TABLE users (id INT AUTO_INCREMENT PRIMARY KEY, login VARCHAR(50) NOT NULL U...
7	16:38:57	CREATE TABLE requests (id INT AUTO_INCREMENT PRIMARY KEY, client_id INT NOT NULL, title...
8	16:38:57	INSERT INTO users (login, password, role) VALUES ('manager', '1234', 'manager'), ('user1', '1234', 'client'), ('us...
9	16:38:57	INSERT INTO requests (client_id, title, description, status) VALUES (2, 'Бронирование ПК', 'Хочу заброниро...

Рисунок 6 – Результат выполнения SQL-скрипта

Поле role объявлено как ENUM, а не VARCHAR: такой тип не позволяет сохранить произвольное значение, отличное от заданного набора, что дополнительно защищает данные от некорректного ввода. Поле login имеет ограничение UNIQUE, исключающее регистрацию двух пользователей с одинаковым именем. Директива ON DELETE CASCADE в таблице requests обеспечивает автоматическое удаление заявок при удалении связанного пользователя – целостность базы при этом не нарушается.

После создания структуры корректность связей была проверена с помощью JOIN-запросов. Все запросы возвращали ожидаемые результаты.

2.2.2 Разработка приложения

Приложение реализовано на языке C# с использованием Windows Forms в среде Microsoft Visual Studio. Оно включает четыре формы, каждая из которых отвечает за определенную область функциональности: LoginForm – авторизация, RegisterForm – регистрация, ManagerForm – личный кабинет менеджера, ClientForm – личный кабинет клиента.

Полный программный код представлен в приложении А.

2.2.2.1 Реализация клиентской части

LoginForm – форма авторизации. Пользователь вводит логин и пароль, выбирает роль из выпадающего списка. После нажатия кнопки «Войти» приложение устанавливает соединение с базой данных и выполняет поиск записи с указанными учетными данными. Если логин и пароль совпали, но роль не соответствует сохраненной, доступ не предоставляется. При успешной

проверке открывается соответствующий рабочий кабинет. Форма авторизации представлена на рисунке 7.

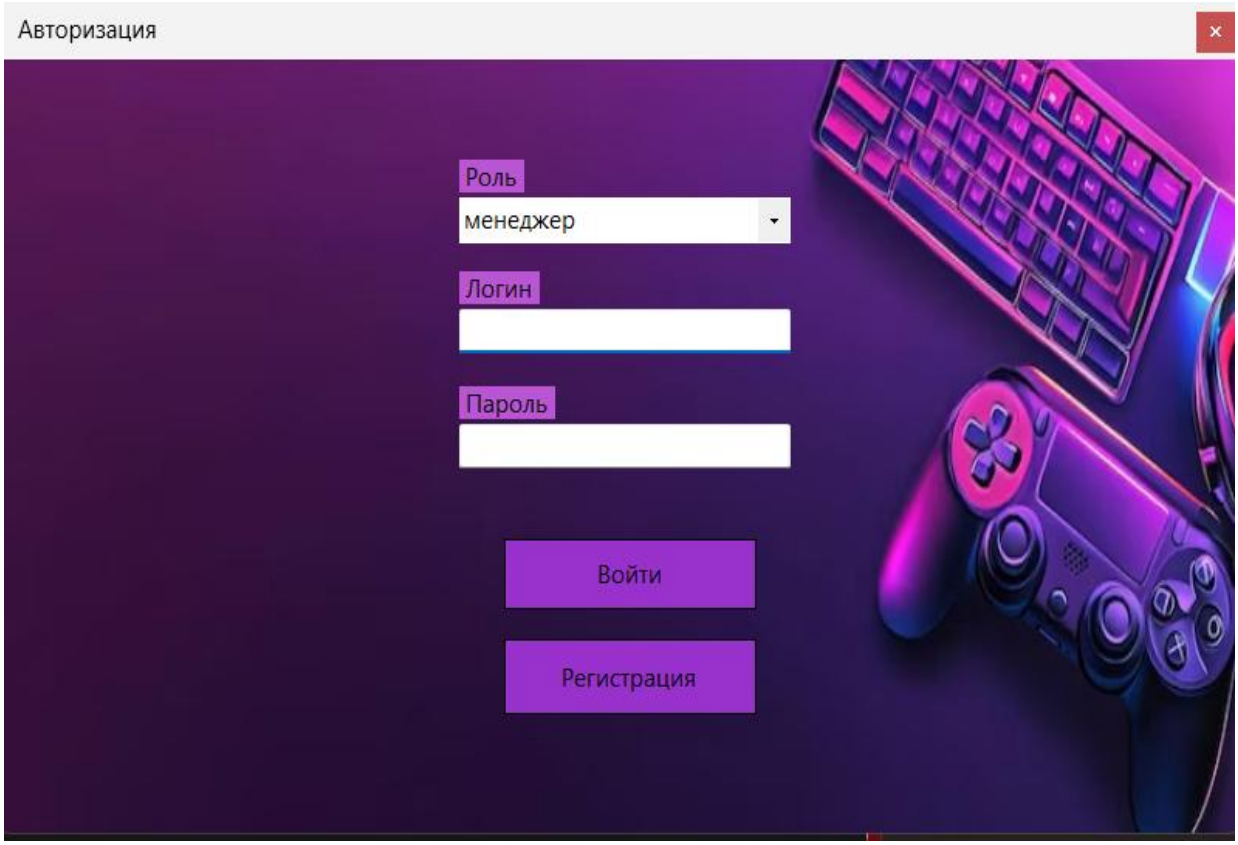

Рисунок 7 – Форма авторизации

RegisterForm – форма регистрации. Новый пользователь вводит логин, пароль и выбирает роль. При попытке зарегистрировать уже занятый логин MySQL возвращает исключение Duplicate entry: приложение перехватывает его и выводит понятное сообщение. Форма регистрации представлена на рисунке 8.

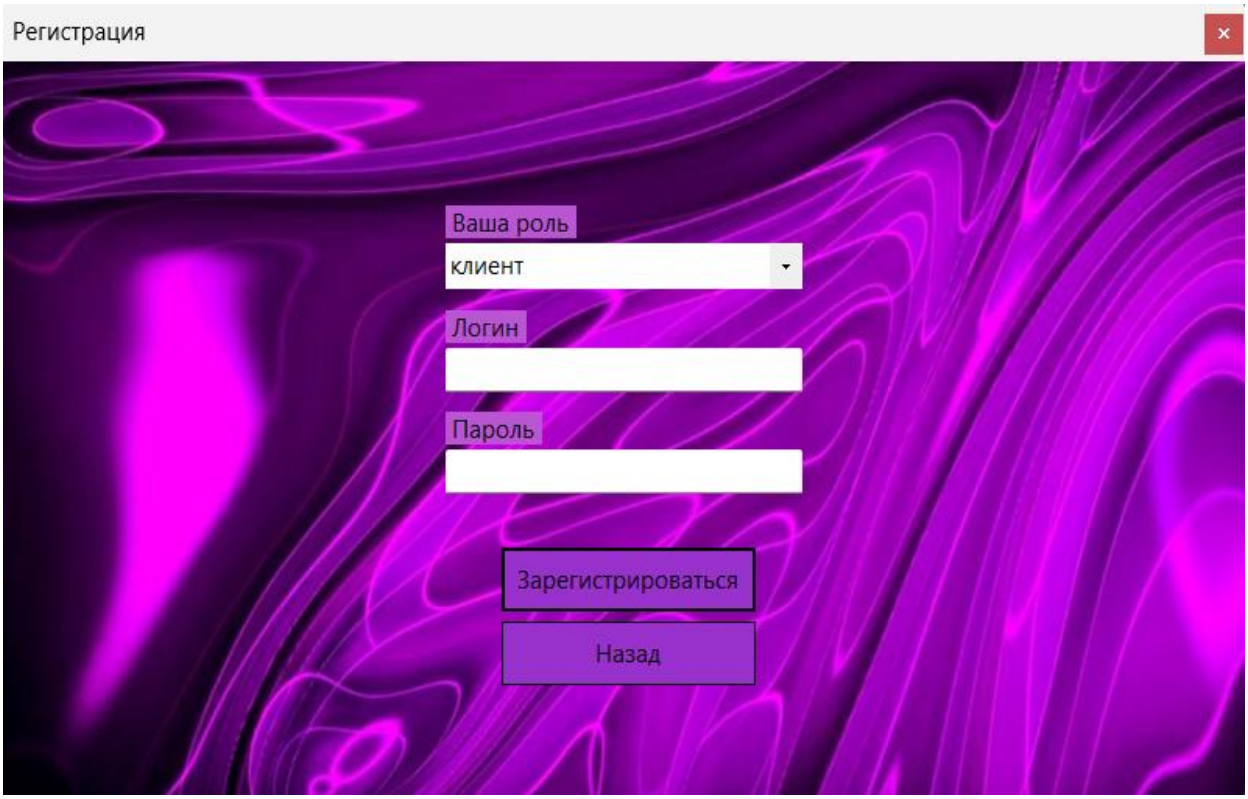

Рисунок 8 – Форма регистрации

ClientForm – личный кабинет клиента. При открытии формы в нее передается идентификатор текущего пользователя (currentUserId). Все SQL-запросы содержат условие WHERE client_id=@id, благодаря чему клиент видит только собственные заявки. На форме расположены поле поиска по названию и выпадающий список для фильтрации по статусу – оба элемента обновляют список заявок в режиме реального времени. Кнопка «Добавить заявку» открывает последовательные диалоговые окна для ввода названия и описания. Личный кабинет клиента представлен на рисунке 9.

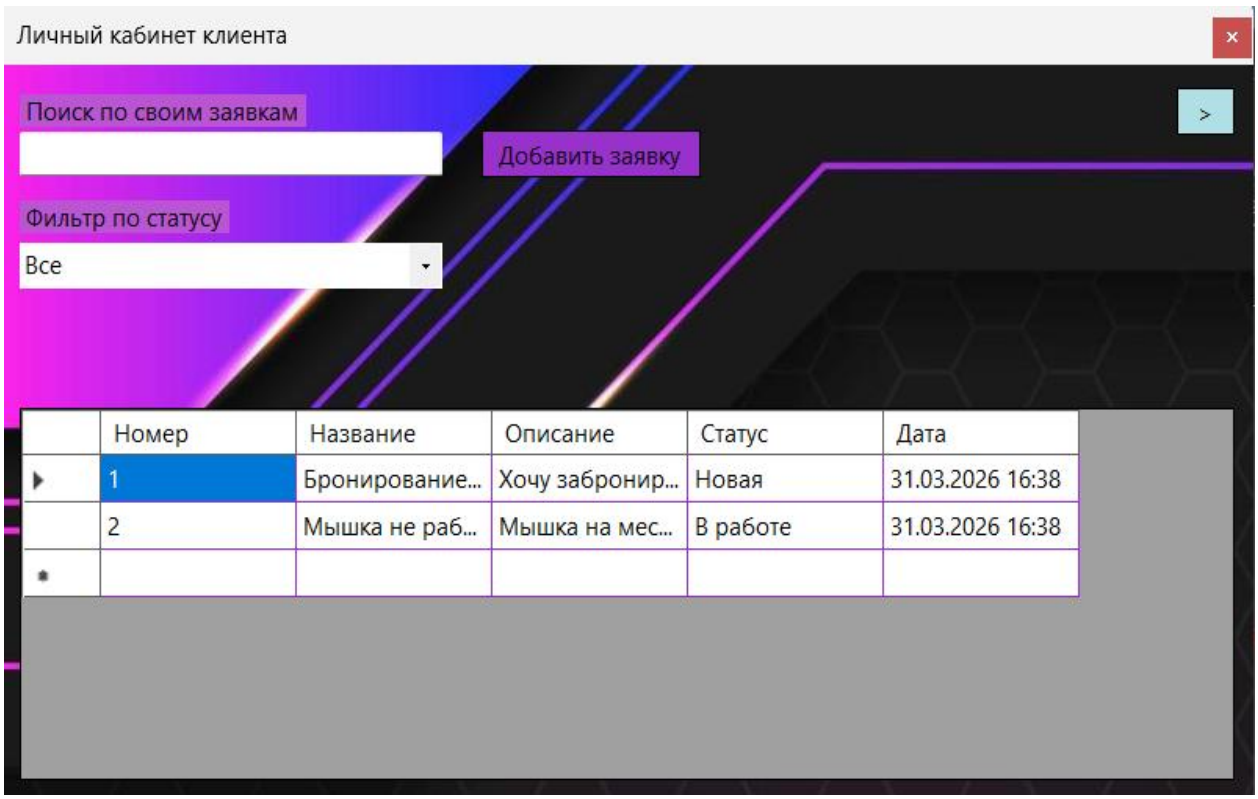

Рисунок 9 – Личный кабинет клиента

ManagerForm – личный кабинет менеджера. Данная форма открывается после входа пользователя с ролью менеджера и предназначена для работы со всеми заявками системы. В отличие от клиента, здесь отсутствует ограничение по пользователю, поэтому используется объединение таблиц JOIN users u ON r.client_id = u.id, что позволяет выводить данные о заявке вместе с логином клиента. При загрузке формы выполняется выборка записей и их отображение в таблице. На форме реализованы поиск по названию, фильтрация по статусу и выбор клиента, при этом в запрос добавляются соответствующие условия, например r.status=@status и r.client_id=@id. Кнопка «Добавить» позволяет создать новую заявку с указанием клиента, изменение статуса выполняется через запрос UPDATE requests SET status=@status WHERE id=@id, а удаление – через DELETE FROM requests WHERE id=@id с подтверждением действия. Личный кабинет менеджера представлен на рисунке 10.

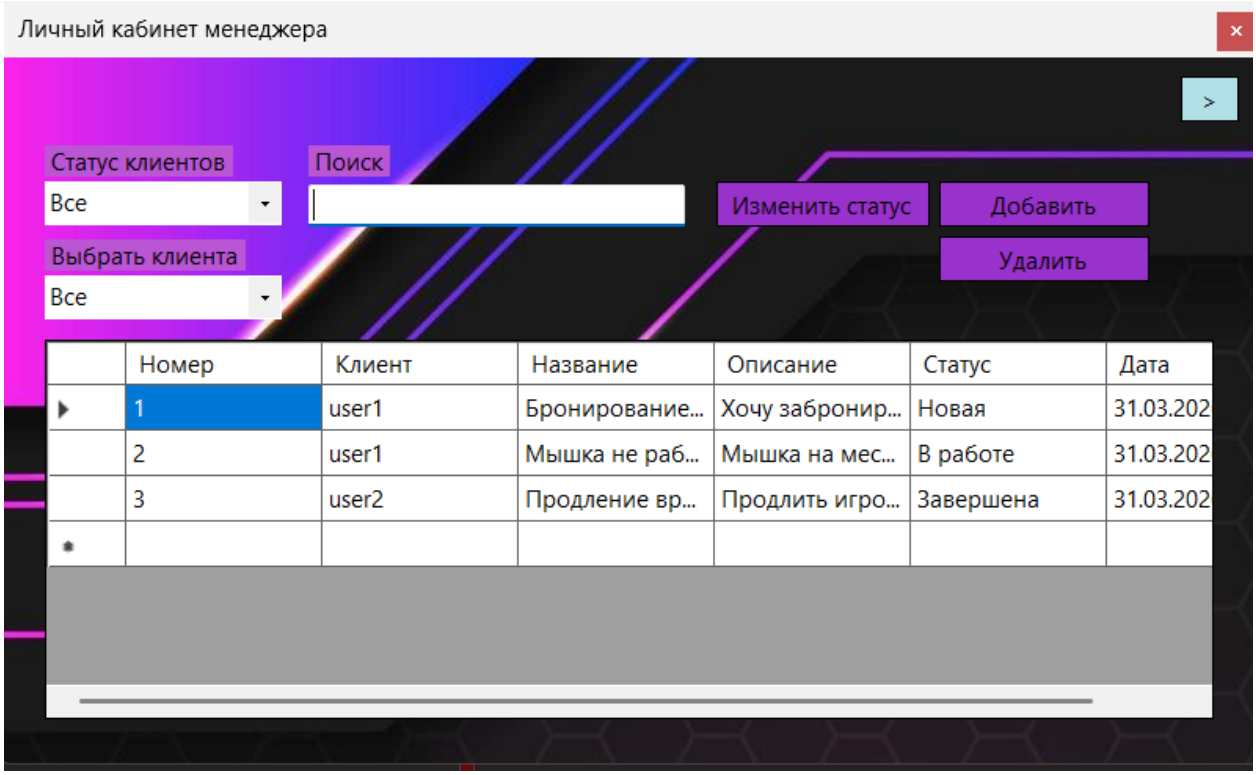

Рисунок 10 – Личный кабинет менеджера

Таким образом, разработанные формы позволяют пользователю удобно работать с системой, при этом доступ к функциям определяется его ролью, что делает использование приложения более понятным и логичным.

2.2.2.2 Разработка серверной части

Обработка данных в приложении осуществляется за счёт взаимодействия с базой данных MySQL. При этом практически любое действие пользователя, будь то вход в систему или работа с заявками, так или иначе связано с выполнением SQL-запросов, через которые происходит обращение к хранимой информации.

При авторизации формируется запрос, позволяющий проверить наличие пользователя с введёнными данными. Если совпадение найдено, дополнительно определяется его роль, поскольку именно от неё зависит дальнейшая работа приложения и выбор интерфейса. В случае несоответствия данных доступ к системе не предоставляется. Процесс проверки введённых данных при входе в систему представлен на рисунке 11.

```
MySqlCommand cmd = new MySqlCommand("SELECT id, role FROM users WHERE login=@login AND password=@pass", con);
cmd.Parameters.AddWithValue("@login", textBoxLogin.Text);
cmd.Parameters.AddWithValue("@pass", textBoxPassword.Text);

int userId = 0;
string actualRole = "";

using (MySqlDataReader reader = cmd.ExecuteReader())
{
    if (reader.Read())
    {
        userId = Convert.ToInt32(reader["id"]);
        actualRole = reader["role"].ToString();
    }
}

if (actualRole == "")
{
    MessageBox.Show("Неверный логин или пароль!");
    return;
}
```

Рисунок 11 – Проверка данных при входе

Регистрация нового пользователя выполняется путём добавления записи в таблицу базы данных. Несмотря на простоту самой операции, в процессе предусмотрена обработка возможных ошибок. В частности, контролируется ситуация, при которой пользователь пытается использовать уже существующий логин. В таком случае выполнение запроса прерывается, а система уведомляет пользователя о невозможности завершения регистрации. Реализация процесса регистрации пользователя показана на рисунке 12.

```
MySqlCommand cmd = new MySqlCommand("INSERT INTO users (login, password, role) VALUES (@login, @pass, @role)", con);
cmd.Parameters.AddWithValue("@login", textBoxLogin.Text);
cmd.Parameters.AddWithValue("@pass", textBoxPassword.Text);
cmd.Parameters.AddWithValue("@role", roleValue);
cmd.ExecuteNonQuery();
```

Рисунок 12 – Добавление пользователя

Добавление заявок реализовано по схожему принципу, однако здесь уже учитываются данные, вводимые пользователем в процессе работы с системой. После сохранения информация сразу становится доступной для просмотра, так как выполняется обновление таблицы. Это позволяет не выполнять дополнительных действий для получения актуальных данных. Пример сохранения заявки приведён на рисунке 13.

```
MySqlCommand cmd = new MySqlCommand("INSERT INTO requests (client_id, title, description, status) VALUES (@id, @title, @desc, 'New')", con);
cmd.Parameters.AddWithValue("@id", currentUserId);
cmd.Parameters.AddWithValue("@title", title);
cmd.Parameters.AddWithValue("@desc", desc);
cmd.ExecuteNonQuery();
```

Рисунок 13 – Сохранение заявки

Получение информации из базы данных осуществляется с учётом параметров, задаваемых пользователем. Например, при вводе текста в поле поиска или выборе определённого статуса формируется уточнённый SQL-запрос. В

результате этого в таблице отображаются только те записи, которые соответствуют указанным условиям, что значительно упрощает работу при большом объёме данных. Важно отметить, что подобный подход позволяет избежать лишней нагрузки и делает интерфейс более отзывчивым. Формирование запроса с учётом заданных условий иллюстрируется на рисунке 14.

```
string sql = "SELECT id AS 'Номер', title AS 'Название', description AS 'Описание', " +
             "CASE status WHEN 'New' THEN 'Новая' WHEN 'In_progress' THEN 'В работе' WHEN 'Completed' THEN 'Завершена' END AS 'Статус', " +
             "created_date AS 'Дата' " +
             "FROM requests WHERE client_id=@id";

if (!string.IsNullOrWhiteSpace(txtSearch.Text))
    sql += " AND title LIKE @search";

if (cmbStatus.Text != "Все")
    sql += " AND status=@status";
```

Рисунок 14 – Формирование запроса с условиями

Изменение данных выполняется через обновление уже существующих записей. В рассматриваемом приложении это касается изменения статуса заявки. Перед выполнением операции производится проверка корректности вводимого значения, что необходимо для предотвращения появления некорректных данных в базе. После этого выполняется обновление записи, и информация в таблице становится актуальной. Процесс изменения статуса заявки представлен на рисунке 15.

```
if (dataGridView1.CurrentRow == null) { MessageBox.Show("Выберите заявку!"); return; }

int id = Convert.ToInt32(dataGridView1.CurrentRow.Cells["Номер"].Value);
string status = Microsoft.VisualBasic.Interaction.InputBox("Статус (Новая / В работе / Завершена):", "Изменить", "");
if (string.IsNullOrWhiteSpace(status)) { MessageBox.Show("Вы ничего не ввели!"); return; }

string dbStatus = StatusRuToDb(status);
if (dbStatus == "") { MessageBox.Show("Статус должен быть: Новая, В работе или Завершена!"); return; }

using (MySqlConnection con = new MySqlConnection(conn))
{
    con.Open();
    MySqlCommand cmd = new MySqlCommand("UPDATE requests SET status=@status WHERE id=@id", con);
    cmd.Parameters.AddWithValue("@status", dbStatus);
    cmd.Parameters.AddWithValue("@id", id);
    cmd.ExecuteNonQuery();
}

LoadData();
MessageBox.Show("Статус изменен!");
```

Рисунок 15 – Обновление данных

Удаление записей также осуществляется через обращение к базе данных. При этом перед выполнением операции система запрашивает подтверждение со стороны пользователя. Это позволяет избежать случайного удаления информации и делает работу с приложением более безопасной. Пример удаления записи из базы данных показан на рисунке 16.

```
if (dataGridView1.CurrentRow == null) { MessageBox.Show("Выберите заявку!"); return; }

int id = Convert.ToInt32(dataGridView1.CurrentRow.Cells["Номер"].Value);

if (MessageBox.Show("Удалить заявку?", "Подтверждение", MessageBoxButtons.YesNo) == DialogResult.Yes)
{
    using (MySqlConnection con = new MySqlConnection(conn))
    {
        con.Open();
        MySqlCommand cmd = new MySqlCommand("DELETE FROM requests WHERE id=@id", con);
        cmd.Parameters.AddWithValue("@id", id);
        cmd.ExecuteNonQuery();
    }

    LoadData();
    MessageBox.Show("Заявка удалена!");
}
```

Рисунок 16 – Удаление записи

В процессе разработки была реализована обработка исключений, благодаря чему удаётся избежать сбоев в работе программы. Это особенно важно при взаимодействии с базой данных, поскольку любые ошибки на данном этапе могут привести к некорректной работе системы.

Таким образом, серверная часть приложения обеспечивает корректную обработку данных и поддерживает стабильную работу всей системы в процессе её использования.

Полный программный код всех форм и реализованной логики приведён в приложениях А–Г.

2.3 Организация хранения и доступа к файлам посредством репозитория

На рисунке 17 представлен репозиторий проекта, размещённый на платформе Git[8]. В него загружены как исходный код приложения, так и SQL-скрипт создания базы данных.

[8] Git – это система для управления версиями исходного кода программ. [Электронный ресурс] – URL: https://skillbox.ru/media/code/chto_takoe_git_obyasnyaem_na_skhemakh/ Режим доступа: свободный (дата обращения: 21.04.2026).

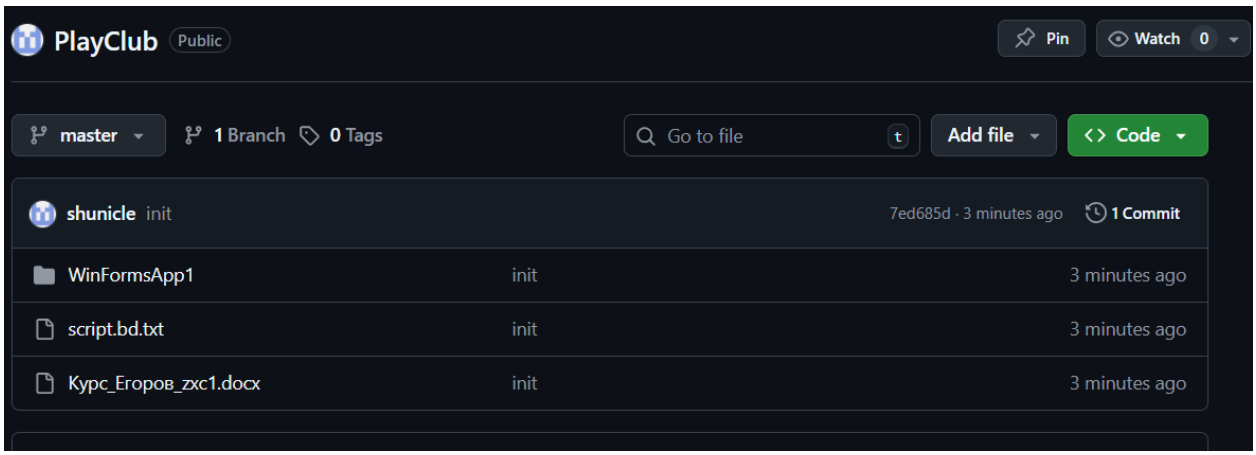

Рисунок 17 – Репозиторий GIT с загруженными проектом и базой данных

Использование системы контроля версий позволяет отслеживать историю изменений: каждое значимое изменение зафиксировано отдельным коммитом с кратким описанием на английском языке. Благодаря этому при необходимости можно откатиться к любой предыдущей версии проекта, не теряя наработанный код.

Репозиторий также выполняет роль резервной копии: даже при потере локального экземпляра проект целиком восстанавливается командой git clone. Доступ к репозиторию открыт, что упрощает проверку и демонстрацию работы.

Проект доступен на репозитории: https://github.com/shunicle/PlayClub .

2.4 Тестирование системы

Цель тестирования – проверить корректность работы всех реализованных функций и убедиться в предсказуемом поведении системы при некорректных действиях пользователя.

Тестирование проводилось вручную по заранее составленным тест-кейсам [5, с. 399–402]. Для каждого сценария определялось предварительное условие, последовательность шагов, ожидаемый и фактический результат. Тестовые сценарии представлены в таблицах 5-12.

Таблица 5 – Тестовый сценарий ТС-01

Тестовый сценарий	ТС-01
Заголовок теста	Авторизация с корректными данными (клиент)
Краткое изложение	Вход в систему с правильными логином, паролем и ролью «клиент»
Предварительное условие	Пользователь user1 существует в БД с ролью «клиент»
Этапы теста	1. Ввести логин: user1. 2. Ввести пароль: 1234. 3. Выбрать роль: клиент. 4. Нажать «Войти».
Ожидаемый результат	Открывается ClientForm (личный кабинет клиента)
Фактический результат	ClientForm открылась, заявки пользователя отображаются
Статус	Зачет
Постусловие	Пользователь находится в личном кабинете
Примечания	Отсутствуют

Таблица 6 – Тестовый сценарий ТС-02

Тестовый сценарий	ТС-02
Заголовок теста	Вход с неверным паролем
Краткое изложение	Попытка авторизации с неправильным паролем
Предварительное условие	Пользователь user1 существует в БД
Этапы теста	1. Ввести логин: user1. 2. Ввести пароль: asdv34z. 3. Нажать «Войти».
Ожидаемый результат	Сообщение «Неверный логин или пароль!», вход не выполнен
Фактический результат	Сообщение отображено, форма авторизации осталась активной
Статус	Зачет
Постусловие	Пользователь остался на форме авторизации
Примечания	Отсутствуют

Таблица 7 – Тестовый сценарий ТС-03

Тестовый сценарий	ТС-03
Заголовок теста	Вход с неверно выбранной ролью
Краткое изложение	Пользователь с ролью «клиент» выбирает роль «менеджер» при авторизации
Предварительное условие	Пользователь user1 зарегистрирован с ролью «клиент»
Этапы теста	1. Ввести логин: user1. 2. Ввести пароль: 1234. 3. Выбрать роль: менеджер. 4. Нажать «Войти».
Ожидаемый результат	Сообщение «Вы выбрали неверную роль!», вход не выполнен
Фактический результат	Сообщение отображено, вход не выполнен
Статус	Зачет
Постусловие	Пользователь остался на форме авторизации
Примечания	Отсутствуют

Таблица 8 – Тестовый сценарий ТС-04

Тестовый сценарий	ТС-04
Заголовок теста	Регистрация нового пользователя
Краткое изложение	Создание новой учетной записи с уникальным логином
Предварительное условие	Логин newuser отсутствует в БД
Этапы теста	1. Перейти на форму регистрации. 2. Ввести логин: newuser. 3. Ввести пароль: pass123. 4. Выбрать роль: клиент. 5. Нажать «Зарегистрироваться».
Ожидаемый результат	Сообщение «Регистрация успешна!», переход на форму входа
Фактический результат	Учетная запись создана, LoginForm открылась
Статус	Зачет
Постусловие	Пользователь newuser добавлен в таблицу users
Примечания	Отсутствуют

Таблица 9 – Тестовый сценарий ТС-05

Тестовый сценарий	ТС-05
Заголовок теста	Регистрация с уже существующим логином
Краткое изложение	Попытка создать учетную запись с занятым логином
Предварительное условие	Пользователь user1 уже существует в БД
Этапы теста	1. Открыть форму регистрации. 2. Ввести логин: user1. 3. Ввести пароль: любой. 4. Нажать «Зарегистрироваться».
Ожидаемый результат	Сообщение «Пользователь с таким логином уже существует!»
Фактический результат	Сообщение отображено, запись в БД не добавлена
Статус	Зачет
Постусловие	Данные в БД не изменились
Примечания	Перехватывается исключение Duplicate entry от MySQL

Таблица 10 – Тестовый сценарий ТС-06

Тестовый сценарий	ТС-06
Заголовок теста	Создание заявки клиентом
Краткое изложение	Добавление новой заявки через личный кабинет клиента
Предварительное условие	Пользователь авторизован с ролью «клиент»
Этапы теста	1. Нажать «Добавить заявку». 2. Ввести название: «Бронирование ПК». 3. Ввести описание: «Хочу место на 2 часа». 4. Подтвердить.
Ожидаемый результат	Заявка отображается в списке со статусом «Новая» и текущей датой
Фактический результат	Заявка добавлена, данные отображаются корректно
Статус	Зачет
Постусловие	Запись добавлена в таблицу requests
Примечания	Отсутствуют

Таблица 11 – Тестовый сценарий ТС-07

Тестовый сценарий	ТС-07
Заголовок теста	Изменение статуса заявки менеджером
Краткое изложение	Менеджер обновляет статус существующей заявки
Предварительное условие	Менеджер авторизован; в системе присутствует заявка со статусом «Новая»
Этапы теста	1. Выбрать заявку в таблице. 2. Нажать «Изменить статус». 3. Ввести: «В работе». 4. Подтвердить.
Ожидаемый результат	Статус заявки изменился на «В работе», таблица обновлена
Фактический результат	Статус изменен, в таблице отображается «В работе»
Статус	Зачет
Постусловие	Запись в таблице requests обновлена
Примечания	Отсутствуют

Таблица 12 – Тестовый сценарий ТС-08

Тестовый сценарий	ТС-08
Заголовок теста	Удаление заявки менеджером
Краткое изложение	Менеджер удаляет заявку из системы
Предварительное условие	Менеджер авторизован; в системе присутствует заявка
Этапы теста	1. Выбрать заявку в таблице. 2. Нажать «Удалить». 3. Подтвердить в диалоге.
Ожидаемый результат	Заявка удалена из списка и из базы данных
Фактический результат	Заявка удалена, список обновился
Статус	Зачет
Постусловие	Запись удалена из таблицы requests
Примечания	Без подтверждения в диалоге удаление не выполняется

Все восемь тестовых сценариев завершились успешно. В ходе тестирования были выявлены и исправлены две ошибки: некорректное обновление списка заявок при изменении фильтра по клиенту и лишний вызов метода LoadData() при инициализации формы менеджера. После внесения исправлений приложение работает стабильно и корректно выполняет все предусмотренные функции.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта было разработано приложение для управления заявками в игровом клубе. Работа охватила полный цикл создания программного продукта – от анализа предметной области до тестирования готовой системы.

Анализ существующей практики показал, что небольшие игровые клубы, ведущие учет заявок вручную, регулярно сталкиваются с потерей данных, ошибками ввода, отсутствием прозрачности по статусам и сложностью поиска нужных записей. Готовые решения – Bitrix24 и 1С: Предприятие – не подходят для данного формата: первое требует значительных затрат на тарифный план, второе – на настройку и обучение.

На этапе проектирования были разработаны структура реляционной базы данных из двух таблиц и диаграмма вариантов использования, определившая набор доступных функций для каждого типа пользователей. Выбранный стек технологий – C#, Windows Forms, MySQL обеспечил возможность реализации всего необходимого функционала без излишней сложности.

Приложение включает следующие возможности:

— Регистрация и авторизация пользователей с разграничением ролей «клиент» и «менеджер»;

— Личный кабинет клиента – создание заявок, просмотр собственных обращений, поиск и фильтрация по статусу;

— Рабочий кабинет менеджера – просмотр всех заявок, добавление, изменение статуса, редактирование, удаление, фильтрация по клиенту и статусу;

— Хранение данных в реляционной базе с контролем целостности посредством внешних ключей и ограничений.

Тестирование по восьми сценариям подтвердило корректность работы всех функций. Выявленные в процессе ошибки были исправлены до финальной сборки.

Цель проекта достигнута: приложение автоматизирует учет заявок в игровом клубе и обеспечивает удобную работу для обеих категорий пользователей.

Среди возможных направлений развития – добавление уведомлений при изменении статуса заявки, реализация веб-версии приложения и расширение ролевой модели.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

Книги под фамилией автора:

1) Горев, А. Э. Информационные технологии в профессиональной деятельности : учебник для среднего профессионального образования / А. Э. Горев. — Москва : Издательство Юрайт, 2018. — 271 с. — (Профес-сиональное образование). — ISBN 978-5-534-10100-3. — Текст : электрон-ный // Образовательная платформа Юрайт [сайт]. — URL: https://urait.ru/bcode/429335 (дата обращения: 11.03.2026).

2) Грекул, В. И. Проектирование информационных систем : учеб-ник и практикум для среднего профессионального образования / В. И. Грекул, Н. Л. Коровкина, Г. А. Левочкина. — 2-е изд. — Москва : Из-дательство Юрайт, 2025. — 404 с. — (Профессиональное образование). — ISBN 978-5-534-19506-4. — Текст : электронный // Образовательная плат-форма Юрайт [сайт]. — URL: https://urait.ru/bcode/566739 (дата обращения: 17.03.2026).

3) Нестеров, С. А. Базы данных : учебник и практикум для среднего профессионального образования / С. А. Нестеров. — 2-е изд. — Москва : Издательство Юрайт, 2026. — 258 с. — (Профессиональное образование). — ISBN 978-5-534-18087-9. — Текст : электронный // Образовательная платформа Юрайт [сайт]. — URL: https://urait.ru/bcode/587742 (дата обращения: 01.04.2026).

4) Экономика отрасли информационных систем : учебник для среднего профессионального образования / А. Л. Рыжко, Н. А. Рыжко, Н. М. Лобанова, Е. О. Кучинская. — 2-е изд., испр. и доп. — Москва : Издательство Юрайт, 2026. — 176 с. — (Профессиональное образование). — ISBN 978-5-534-11628-1. — Текст : электронный // Образовательная платформа Юрайт [сайт]. — URL: https://urait.ru/bcode/587743 (дата обращения: 18.03.2026).

5) Советов, Б. Я. Информационные технологии : учебник для среднего профессионального образования / Б. Я. Советов, В. В. Цехановский. — 8-е изд., перераб. и доп. — Москва : Издательство Юрайт, 2026. — 414 с. — (Профессиональное образование). — ISBN 978-5-534-20053-9. — Текст :

электронный // Образовательная платформа Юрайт [сайт]. — URL: https://urait.ru/bcode/583524 (дата обращения: 20.03.2026).

6) Информационные технологии : учебник для среднего профессионального образования / В. В. Трофимов, О. П. Ильина, В. И. Кияев, Е. В. Трофимова ; ответственный редактор В. В. Трофимов. — Москва : Из-дательство Юрайт, 2026. — 546 с. — (Профессиональное образование). — ISBN 978-5-534-18341-2. — Текст : электронный // Образовательная плат-форма Юрайт [сайт]. — URL: https://urait.ru/bcode/589573 (дата обращения: 22.04.2026).

7) Проектирование информационных систем : учебник и практикум для среднего профессионального образования / Д. В. Чистов, П. П. Мельников, А. В. Золотарюк, Н. Б. Ничепорук. — 2-е изд., перераб. и доп. — Москва : Издательство Юрайт, 2026. — 273 с. — (Профессиональ-ное образование). — ISBN 978-5-534-20362-2. — Текст : электронный // Образовательная платформа Юрайт [сайт]. — URL: https://urait.ru/bcode/584914 (дата обращения: 08.04.2026).

Электронные ресурсы:

8) CRM-система – это функциональный инструмент, который помогает выстраивать и поддерживать отношения с клиентами. [Электронный ресурс] – URL: https://crm.ru/blog/crm-chto-eto-takoe/ Режим доступа: свободный (дата обращения: 12.03.2026).

9) ER-диаграмма (Entity-Relationship, или диаграмма «сущность-связь») — это схема, которая показывает, какие данные есть в системе и как они связаны друг с другом. [Электронный ресурс] – URL: https://kurshub.ru/journal/blog/chto-takoe-er-diagramma/ Режим доступа: свободный (дата обращения: 01.04.2026).

10) Git – это система для управления версиями исходного кода программ. [Электронный ресурс] – URL: https://skillbox.ru/media/code/chto_takoe_git_obyasnyaem_na_skhemakh/ Режим доступа: свободный (дата обращения: 21.04.2026).

11) LINQ (Language Integrated Query) – это набор методов для работы с коллекциями. [Электронный ресурс] – URL: https://habr.com/ru/articles/836752/ Режим доступа: свободный (дата обращения: 31.03.2026).

12) MySQL (My Structured Query Language) – реляционная система управления базами данных (СУБД) с открытым исходным кодом, основанная на языке SQL.[Электронный ресурс] – URL: https://selectel.ru/blog/mysql / Режим доступа: свободный (дата обращения: 09.04.2026).

13) SQL-запросы — Structured Query Language предназначен для управления базами данных. Чтобы обрабатывать данные из базы, нужно использовать SQL-запросы. [Электронный ресурс] – URL: https://practicum.yandex.ru/blog/sql-zaprosy-dlya-upravleniya-bazami-dannyh / / Режим доступа: свободный (дата обращения: 09.04.2026).

14) WPF (Windows Presentation Foundation) — это библиотека для создания пользовательских интерфейсов для интеллектуальных клиентских приложений. [Электронный ресурс] – URL: https://professorweb.ru/my/WPF/base_WPF/level1/base_WPF_index.php / Режим доступа: свободный (дата обращения: 31.03.2026).

15) Windows Forms (WinForms) – это платформа пользовательского интерфейса для создания классических приложений Windows. [Электронный ресурс] – URL: https://learn.microsoft.com/ru-ru/dotnet/desktop/winforms/overview/ Режим доступа: свобод-ный (дата обращения: 31.03.2026).

ПРИЛОЖЕНИЯ

ПРИЛОЖЕНИЕ А

Программный код страницы «Авторизация»

```
using MySql.Data.MySqlClient;
using System;
using System.Windows.Forms;

namespace WinFormsApp1
{
    public partial class LoginForm : Form
    {
        string conn = "server=localhost;database=testdb;uid=root;pwd=1234;";

        public LoginForm()
        {
            InitializeComponent();
            comboBoxRole.Items.Clear();
            comboBoxRole.Items.Add("менеджер");
            comboBoxRole.Items.Add("клиент");
            comboBoxRole.SelectedIndex = 0;
        }

        protected override void OnFormClosed(FormClosedEventArgs e)
        {
            base.OnFormClosed(e);
            Application.Exit();
        }

        private void buttonLogin_Click(object sender, EventArgs e)
        {
            if (string.IsNullOrWhiteSpace(textBoxLogin.Text) || string.IsNullOr-
WhiteSpace(textBoxPassword.Text))
            {
                MessageBox.Show("Заполните логин и пароль!");
                return;
            }

            if (comboBoxRole.SelectedIndex == -1)
            {
                MessageBox.Show("Выберите роль!");
                return;
            }

            try
            {
                using (MySqlConnection con = new MySqlConnection(conn))
                {
                    con.Open();

                    MySqlCommand cmd = new MySqlCommand("SELECT id, role FROM users
WHERE login=@login AND password=@pass", con);
                    cmd.Parameters.AddWithValue("@login", textBoxLogin.Text);
                    cmd.Parameters.AddWithValue("@pass", textBoxPassword.Text);

                    int userId = 0;
                    string actualRole = "";

                    using (MySqlDataReader reader = cmd.ExecuteReader())
                    {
                        if (reader.Read())
                        {
                            userId = Convert.ToInt32(reader["id"]);
                            actualRole = reader["role"].ToString();
                        }
                    }
```

```
                    if (actualRole == "")
                    {
                        MessageBox.Show("Неверный логин или пароль!");
                        return;
                    }

                    string selectedRole = comboBoxRole.Text == "менеджер" ? "manager" :
"client";

                    if (actualRole != selectedRole)
                    {
                        MessageBox.Show("Вы выбрали неверную роль!");
                        return;
                    }

                    if (actualRole == "manager")
                    {
                        ManagerForm managerForm = new ManagerForm();
                        managerForm.Show();
                    }
                    else
                    {
                        ClientForm clientForm = new ClientForm(userId);
                        clientForm.Show();
                    }

                    this.Hide();
                }
            }
            catch (Exception ex)
            {
                MessageBox.Show("Ошибка подключения: " + ex.Message);
            }
        }

        private void buttonRegister_Click(object sender, EventArgs e)
        {
            RegisterForm registerForm = new RegisterForm();
            registerForm.Show();
            this.Hide();
        }
    }
}
```

ПРИЛОЖЕНИЕ Б

Программный код формы «Регистрация»

```
using MySql.Data.MySqlClient;
using System;
using System.Windows.Forms;

namespace WinFormsApp1
{
    public partial class RegisterForm : Form
    {
        string conn = "server=localhost;database=testdb;uid=root;pwd=1234;";

        public RegisterForm()
        {
            InitializeComponent();
            comboBoxRole1.Items.Clear();
            comboBoxRole1.Items.Add("клиент");
            comboBoxRole1.Items.Add("менеджер");
            comboBoxRole1.SelectedIndex = 0;
        }

        private void buttonReg_Click(object sender, EventArgs e)
        {
            if (string.IsNullOrWhiteSpace(textBoxLogin.Text) || string.IsNullOr-
WhiteSpace(textBoxPassword.Text))
            {
                MessageBox.Show("Заполните логин и пароль!");
                return;
            }

            string roleValue = comboBoxRole1.Text == "менеджер" ? "manager" : "client";

            try
            {
                using (MySqlConnection con = new MySqlConnection(conn))
                {
                    con.Open();
                    MySqlCommand cmd = new MySqlCommand("INSERT INTO users (login, pass-
word, role) VALUES (@login, @pass, @role)", con);
                    cmd.Parameters.AddWithValue("@login", textBoxLogin.Text);
                    cmd.Parameters.AddWithValue("@pass", textBoxPassword.Text);
                    cmd.Parameters.AddWithValue("@role", roleValue);
                    cmd.ExecuteNonQuery();
                }

                MessageBox.Show("Регистрация успешна!");
                LoginForm loginForm = new LoginForm();
                loginForm.Show();
                this.Hide();
            }
            catch (Exception ex)
            {
                if (ex.Message.Contains("Duplicate entry"))
                    MessageBox.Show("Пользователь с таким логином уже существует!");
                else
                    MessageBox.Show("Ошибка: " + ex.Message);
            }
        }

        private void button1_Click(object sender, EventArgs e)
        {
            LoginForm loginForm = new LoginForm();
            loginForm.Show();
            this.Hide();
        }
    }
}
```

ПРИЛОЖЕНИЕ В

Программный код формы «Личный кабинет клиента»

```
using MySql.Data.MySqlClient;
using System;
using System.Data;
using System.Windows.Forms;

namespace WinFormsApp1
{
    public partial class ClientForm : Form
    {
        string conn = "server=localhost;database=testdb;uid=root;pwd=1234;";
        int currentUserId;

        public ClientForm(int userId)
        {
            InitializeComponent();
            currentUserId = userId;

            cmbStatus.Items.Clear();
            cmbStatus.Items.Add("Все");
            cmbStatus.Items.Add("Новая");
            cmbStatus.Items.Add("В работе");
            cmbStatus.Items.Add("Завершена");
            cmbStatus.SelectedIndex = 0;

            LoadData();
        }

        protected override void OnFormClosed(FormClosedEventArgs e)
        {
            base.OnFormClosed(e);
            Application.Exit();
        }

        string StatusRuToDb(string ru)
        {
            if (ru == "Новая") return "New";
            if (ru == "В работе") return "In_progress";
            if (ru == "Завершена") return "Completed";
            return "";
        }

        void LoadData()
        {
            string sql = "SELECT id AS 'Номер', title AS 'Название', description AS
'Описание', " +
                         "CASE status WHEN 'New' THEN 'Новая' WHEN 'In_progress' THEN 'В
работе' WHEN 'Completed' THEN 'Завершена' END AS 'Статус', " +
                         "created_date AS 'Дата' " +
                         "FROM requests WHERE client_id=@id";

            if (!string.IsNullOrWhiteSpace(txtSearch.Text))
                sql += " AND title LIKE @search";

            if (cmbStatus.Text != "Все")
                sql += " AND status=@status";

            DataTable dt = new DataTable();

            using (MySqlConnection con = new MySqlConnection(conn))
            {
                con.Open();
                MySqlCommand cmd = new MySqlCommand(sql, con);
                cmd.Parameters.AddWithValue("@id", currentUserId);

                if (!string.IsNullOrWhiteSpace(txtSearch.Text))
```

```
                    cmd.Parameters.AddWithValue("@search", "%" + txtSearch.Text + "%");

                if (cmbStatus.Text != "Все")
                    cmd.Parameters.AddWithValue("@status", StatusRuToDb(cmbSta-
tus.Text));

                new MySqlDataAdapter(cmd).Fill(dt);
            }

            dgvMyRequests.DataSource = dt;
        }

        private void cmbStatus_SelectedIndexChanged(object sender, EventArgs e) => Load-
Data();

        private void txtSearch_TextChanged_1(object sender, EventArgs e) => LoadData();

        private void btnAddRequest_Click(object sender, EventArgs e)
        {
            string title = Microsoft.VisualBasic.Interaction.InputBox("Название:",
"Добавить заявку", "");
            if (string.IsNullOrWhiteSpace(title)) { MessageBox.Show("Вы ничего не
ввели!"); return; }

            string desc = Microsoft.VisualBasic.Interaction.InputBox("Описание:",
"Добавить заявку", "");
            if (string.IsNullOrWhiteSpace(desc)) { MessageBox.Show("Вы ничего не
ввели!"); return; }

            using (MySqlConnection con = new MySqlConnection(conn))
            {
                con.Open();
                MySqlCommand cmd = new MySqlCommand("INSERT INTO requests (client_id,
title, description, status) VALUES (@id, @title, @desc, 'New')", con);
                cmd.Parameters.AddWithValue("@id", currentUserId);
                cmd.Parameters.AddWithValue("@title", title);
                cmd.Parameters.AddWithValue("@desc", desc);
                cmd.ExecuteNonQuery();
            }

            LoadData();
        }

        private void button2_Click_1(object sender, EventArgs e)
        {
            LoginForm zxc = new LoginForm();
            zxc.Show();
            this.Hide();
        }
    }
      }
```

ПРИЛОЖЕНИЕ Г

Программный код формы «Личный кабинет менеджера»

```
using MySql.Data.MySqlClient;
using System;
using System.Data;
using System.Windows.Forms;

namespace WinFormsApp1
{
    public partial class ManagerForm : Form
    {
        string conn = "server=localhost;database=testdb;uid=root;pwd=1234;";

        public ManagerForm()
        {
            InitializeComponent();
            LoadStatusFilter();
            LoadClients();
            LoadData();
        }

        protected override void OnFormClosed(FormClosedEventArgs e)
        {
            base.OnFormClosed(e);
            Application.Exit();
        }

        void LoadStatusFilter()
        {
            cmbStatus.Items.Clear();
            cmbStatus.Items.Add("Все");
            cmbStatus.Items.Add("Новая");
            cmbStatus.Items.Add("В работе");
            cmbStatus.Items.Add("Завершена");
            cmbStatus.SelectedIndex = 0;
        }

        void LoadClients()
        {
            DataTable dt = new DataTable();
            using (MySqlConnection con = new MySqlConnection(conn))
            {
                new MySqlDataAdapter("SELECT id, login FROM users WHERE role='client'",
con).Fill(dt);
            }

            DataRow allRow = dt.NewRow();
            allRow["id"] = -1;
            allRow["login"] = "Все";
            dt.Rows.InsertAt(allRow, 0);

            cmbClientFilter.DataSource = dt;
            cmbClientFilter.DisplayMember = "login";
            cmbClientFilter.ValueMember = "id";
            cmbClientFilter.SelectedIndex = 0;
        }

        string StatusRuToDb(string ru)
        {
            if (ru == "Новая") return "New";
            if (ru == "В работе") return "In_progress";
            if (ru == "Завершена") return "Completed";
            return "";
        }
```

```
        void LoadData()
        {
            string sql = "SELECT r.id AS 'Номер', u.login AS 'Клиент', r.title AS
'Название', r.description AS 'Описание', " +
                         "CASE r.status WHEN 'New' THEN 'Новая' WHEN 'In_progress' THEN
'В работе' WHEN 'Completed' THEN 'Завершена' END AS 'Статус', " +
                         "r.created_date AS 'Дата' " +
                         "FROM requests r JOIN users u ON r.client_id = u.id WHERE 1=1";

            if (!string.IsNullOrWhiteSpace(txtSearch.Text))
                sql += " AND r.title LIKE '%" + txtSearch.Text + "%'";

            if (cmbStatus.Text != "Все")
                sql += " AND r.status='" + StatusRuToDb(cmbStatus.Text) + "'";

            if (cmbClientFilter.SelectedValue is int clientId && clientId != -1)
                sql += " AND r.client_id=" + clientId;

            DataTable dt = new DataTable();
            new MySqlDataAdapter(sql, conn).Fill(dt);
            dataGridView1.DataSource = dt;
        }

        private void txtSearch_TextChanged_1(object sender, EventArgs e) => LoadData();

        private void cmbStatus_SelectedIndexChanged(object sender, EventArgs e) => Load-
Data();

        private void cmbClientFilter_SelectedIndexChanged(object sender, EventArgs e) =>
LoadData();

        private void btnAdd_Click(object sender, EventArgs e)
        {
            if (!(cmbClientFilter.SelectedValue is int clientId) || clientId == -1)
            {
                MessageBox.Show("Выберите конкретного клиента!");
                return;
            }

            string title = Microsoft.VisualBasic.Interaction.InputBox("Название:",
"Добавить", "");
            if (string.IsNullOrWhiteSpace(title)) { MessageBox.Show("Вы ничего не
ввели!"); return; }

            string desc = Microsoft.VisualBasic.Interaction.InputBox("Описание:",
"Добавить", "");
            if (string.IsNullOrWhiteSpace(desc)) { MessageBox.Show("Вы ничего не
ввели!"); return; }

            using (MySqlConnection con = new MySqlConnection(conn))
            {
                con.Open();
                MySqlCommand cmd = new MySqlCommand("INSERT INTO requests (client_id,
title, description, status) VALUES (@id, @title, @desc, 'New')", con);
                cmd.Parameters.AddWithValue("@id", clientId);
                cmd.Parameters.AddWithValue("@title", title);
                cmd.Parameters.AddWithValue("@desc", desc);
                cmd.ExecuteNonQuery();
            }

            LoadData();
        }

        private void btnUpdateStatus_Click(object sender, EventArgs e)
        {
            if (dataGridView1.CurrentRow == null) { MessageBox.Show("Выберите заявку!");
return; }

            int id = Convert.ToInt32(dataGridView1.CurrentRow.Cells["Номер"].Value);
```

```
            string status = Microsoft.VisualBasic.Interaction.InputBox("Статус (Новая /
В работе / Завершена):", "Изменить", "");
            if (string.IsNullOrWhiteSpace(status)) { MessageBox.Show("Вы ничего не
ввели!"); return; }

            string dbStatus = StatusRuToDb(status);
            if (dbStatus == "") { MessageBox.Show("Статус должен быть: Новая, В работе
или Завершена!"); return; }

            using (MySqlConnection con = new MySqlConnection(conn))
            {
                con.Open();
                MySqlCommand cmd = new MySqlCommand("UPDATE requests SET status=@status
WHERE id=@id", con);
                cmd.Parameters.AddWithValue("@status", dbStatus);
                cmd.Parameters.AddWithValue("@id", id);
                cmd.ExecuteNonQuery();
            }

            LoadData();
            MessageBox.Show("Статус изменен!");
        }

        private void btnDelete_Click(object sender, EventArgs e)
        {
            if (dataGridView1.CurrentRow == null) { MessageBox.Show("Выберите заявку!");
return; }

            int id = Convert.ToInt32(dataGridView1.CurrentRow.Cells["Номер"].Value);

            if (MessageBox.Show("Удалить заявку?", "Подтверждение", MessageBoxBut-
tons.YesNo) == DialogResult.Yes)
            {
                using (MySqlConnection con = new MySqlConnection(conn))
                {
                    con.Open();
                    MySqlCommand cmd = new MySqlCommand("DELETE FROM requests WHERE
id=@id", con);
                    cmd.Parameters.AddWithValue("@id", id);
                    cmd.ExecuteNonQuery();
                }

                LoadData();
                MessageBox.Show("Заявка удалена!");
            }
        }

        private void button2_Click_1(object sender, EventArgs e)
        {
            LoginForm loginForm = new LoginForm();
            loginForm.Show();
            this.Hide();
        }
    }
}
```

ПРИЛОЖЕНИЕ Д

SQL-скрипт создания базы данных и заполнения тестовыми данными

```
DROP DATABASE IF EXISTS testdb;
CREATE DATABASE testdb CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;
USE testdb;

CREATE TABLE users (
   id INT AUTO_INCREMENT PRIMARY KEY,
   login VARCHAR(50) NOT NULL UNIQUE,
   password VARCHAR(100) NOT NULL,
   role ENUM('manager', 'client') NOT NULL DEFAULT 'client',
   reg_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE requests (
   id INT AUTO_INCREMENT PRIMARY KEY,
   client_id INT NOT NULL,
   title VARCHAR(255) NOT NULL,
   description TEXT,
   status ENUM('New', 'In_progress', 'Completed') DEFAULT 'New',
   created_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
   FOREIGN KEY (client_id) REFERENCES users(id) ON DELETE CASCADE
);

INSERT INTO users (login, password, role) VALUES
('manager', '1234', 'manager'),
('user1', '1234', 'client'),
('user2', '1234', 'client');
```

```
INSERT INTO requests (client_id, title, description, status) VALUES
(2, 'Бронирование ПК', 'Хочу забронировать компьютер на 3 часа', 'New'),
(2, 'Мышка не работает', 'Мышка на месте 5 не работает', 'In_progress'),
(3, 'Продление времени', 'Продлить игровое время на 1 час', 'Completed');
```